

Design Simulation Model

User Guide



Design Simulation Model

User Guide

Copyright © 2005 ARM Limited. All rights reserved.

Release Information

The table titled *Release history* lists the changes that have been made to this document.

Release history		
Date	Issue	Change
20 January 2005	A	First release

Proprietary Notice

Words and logos marked with ® or ™ are registered trademarks or trademarks of ARM Limited in the EU and other countries, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by ARM and the party that ARM delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

<http://www.arm.com>

Contents

Design Simulation Model User Guide

	Preface	
	About this guide	x
	Feedback	xiii
Chapter 1	Introduction	
	1.1 About DSMs	1-2
	1.2 Simulation with DSMs	1-3
Chapter 2	Out Of Box Procedures	
	2.1 Installation	2-2
	2.2 Directory structure and files	2-3
	2.3 Licensing	2-6
	2.4 Install the SWIFT model libraries	2-9
	2.5 Testing your model installation	2-10
Chapter 3	Configuring a Model	
	3.1 Setting up your model	3-2
	3.2 Configuring the HDL wrapper	3-3
	3.3 Configuring the model manager	3-4
	3.4 Configuring the DSM	3-10

Chapter 4	Using a Model	
4.1	Facilities for debugging a simulation	4-2
4.2	Limitations of use	4-6
Chapter 5	Timing	
5.1	About the timing shell	5-2
5.2	Multiple timing paths	5-8
5.3	SDF annotation	5-10
5.4	Interconnect delays	5-18
5.5	Use of negative timing checks	5-20
5.6	STA and HDL annotated simulation differences	5-23
5.7	Limitations of the timing model	5-26
Appendix A	EIS Output Format	
A.1	About the EIS file format	A-2
A.2	ARM7TDMI r4 format	A-3
A.3	ARM7TDMI r4 example	A-5
	Glossary	

List of Tables

Design Simulation Model User Guide

	Release history	ii
Table 3-1	Model environment variables	3-2
Table 4-1	Top-level registers	4-2

List of Figures

Design Simulation Model User Guide

Figure 1-1	DSM integration	1-3
Figure 2-1	Directory structure	2-3
Figure 4-1	Program status register bit assignments	4-3
Figure 5-1	HDL wrapper structure	5-3
Figure 5-2	Simple synchronous block with clock	5-4
Figure 5-3	A modified sequence of events	5-6
Figure 5-4	High performance device with negative setup time	5-7
Figure 5-5	Simple timing shell structure	5-10
Figure 5-6	Negative setup time	5-20
Figure 5-7	Delaying a clock signal to produce a new signal	5-21
Figure 5-8	Inputs sampled beyond hold time	5-21
Figure 5-9	Uncertainty of value	5-26

Preface

This preface introduces the *Design Simulation Model User Guide*. It contains the following sections:

- *About this guide* on page x
- *Feedback* on page xiii.

About this guide

This is the *User Guide* for ARM *Design Simulation Models* (DSMs).

Intended audience

This guide is written for experienced hardware engineers, software engineers and *System-on-Chip* (SoC) designers who might have experience of ARM products. You are expected to have experience of Verilog or VHDL.

Using this guide

This guide is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for a high-level description of DSMs and how they are used in simulations.

Chapter 2 *Out Of Box Procedures*

Read this chapter for a description of how to install, set up, and test a DSM.

Chapter 3 *Configuring a Model*

Read this chapter for a description of how to configure the *Hardware Description Language* (HDL) wrapper, model manager, and DSM for use.

Chapter 4 *Using a Model*

Read this chapter for a description of the timing issues for synthesizable and non-synthesizable cores, the top-level registers, and how to control instruction stream tracing. The chapter also describes some of the limitations that DSMs are subject to.

Chapter 5 *Timing*

Read this chapter for a description of timing issues and some of the facilities that ARM Limited provide to aid your annotation process.

———— **Note** ————

This chapter is applicable for:

- timing sign-off with a non-synthesizable processor
- some preliminary timing simulation with a synthesizable processor.

This chapter is not applicable if you want to do timing sign-off for a synthesizable processor because the DSM is not suitable for this. You must use a dedicated *Sign-Off Model* (SOM) instead.

Appendix A *EIS Output Format*

Read this appendix for a description of the *Executed Instruction Stream* (EIS) file format.

Glossary Read the Glossary for definitions of terms used in this guide.

Conventions

Conventions that this guide can use are described in:

- *Typographical*
- *Numbering* on page xii.

Typographical

The typographical conventions are:

<i>italic</i>	Highlights important notes, introduces special terminology, denotes internal cross-references, and citations.
bold	Highlights interface elements, such as menu names. Denotes signal names. Also used for terms in descriptive lists, where appropriate.
monospace	Denotes text that you can enter at the keyboard, such as commands, file and program names, and source code.
<u>monospace</u>	Denotes a permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<i>monospace italic</i>	Denotes arguments to monospace text where the argument is to be replaced by a specific value.
monospace bold	Denotes language keywords when used outside example code.
< and >	Angle brackets enclose replaceable terms for assembler syntax where they appear in code or code fragments. They appear in normal font in running text. For example: • MRC p15, 0 <Rd>, <CRn>, <CRm>, <Opcode_2> • The Opcode_2 value selects which register is accessed.

Numbering

The numbering convention is:

<size in bits>'<base><number>

This is a Verilog method of abbreviating constant numbers. For example:

- 'h7B4 is an unsized hexadecimal value.
- 'o7654 is an unsized octal value.
- 8'd9 is an eight-bit wide decimal value of 9.
- 8'h3F is an eight-bit wide hexadecimal value of 0x3F. This is equivalent to b00111111.
- 8'b1111 is an eight-bit wide binary value of b00001111.

Further reading

This section lists publications by ARM Limited, and by third parties.

ARM Limited periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets, addenda, and the Frequently Asked Questions list.

ARM publications

This guide contains information that is specific to the DSM. See the following documents for other relevant information:

- *RealView ARMulator ISS User Guide* (ARM DUI 0207)
- *ARM Architecture Reference Manual* (ARM DDI 0100)
- *ADS Debug Target Guide* (ARM DUI 0058).

Other publications

This section lists relevant documents published by third parties:

- IEEE std. 1076.4 - 1995 *IEEE Standard for VITAL Application-Specific Integrated Circuit (ASIC) Modeling Specification*, also known as *VITAL-95*.
- IEEE std. 1364 - 1995 *IEEE Standard Hardware Description Language based on the Verilog Hardware Description Language*.
- IEEE std. 61523-3 - 1995 (1497 - 2001) *Delay and power calculation standards - Part 3: Standard Delay Format for the electronic design process*.

Feedback

ARM Limited welcomes feedback on DSMs and their documentation.

Feedback on this product

If you have any comments or suggestions about this product, contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this guide

If you have any comments on this guide, send email to errata@arm.com giving:

- the title
- the number
- the relevant page number(s) to which your comments apply
- a concise explanation of your comments.

ARM Limited also welcomes general suggestions for additions and improvements.

Chapter 1

Introduction

This chapter provides a high-level description of DSMs and their use in simulation. It contains the following sections:

- *About DSMs* on page 1-2
- *Simulation with DSMs* on page 1-3.

1.1 About DSMs

DSMs are back-annotation-capable, timing accurate, simulation models that you can include in a range of target HDL simulators. Each DSM is specific to a simulator and host platform.

Each DSM matches the architecture and functionality of a specific ARM core design. ARM Limited ensures this by certifying the DSM against the entire *Architecture Validation Suite* (AVS) and *Device Validation Suite* (DVS) associated with that ARM core.

DSMs can function with a wide range of industry-standard Verilog and VHDL simulators. They are derived directly from the ARM core *Register Transfer Level* (RTL) code. They can accept timing data through the *Standard Delay Format* (SDF) annotation facility on the simulator.

DSM execution speeds are in the range of 5-500 *cycles-per-second* (cps), depending on:

- simulator interface efficiency
- the complexity of the design it is instantiated in
- the complexity of the CPU that is being modeled.

Features of DSMs are:

- full device functionality
- phase accuracy
- register visibility
- minimum, typical, and maximum pin-to-pin delays
- setup and hold, input pulse checking, output delays
- cache and memory size configuration
- timing and back-annotation
- nine-value logic and resolution
- disassembler.

Note

DSMs are intended for functional and, in some cases, timing simulation. They are not designed for fault simulation or static timing analysis.

1.2 Simulation with DSMs

Figure 1-1 shows the integration of DSMs into an HDL simulation. During simulation, the DSM interfaces to the simulator through the use of a simulator-specific model manager supplied with the DSM. The model manager is a dynamically loaded library that uses the simulator API to interface between the simulator and the DSMs in the design. The simulator invokes the model manager through the HDL wrapper. This presents a module or entity that consists of the connections to the device, as described in the appropriate device *Technical Reference Manual* (TRM), into the logic simulator.

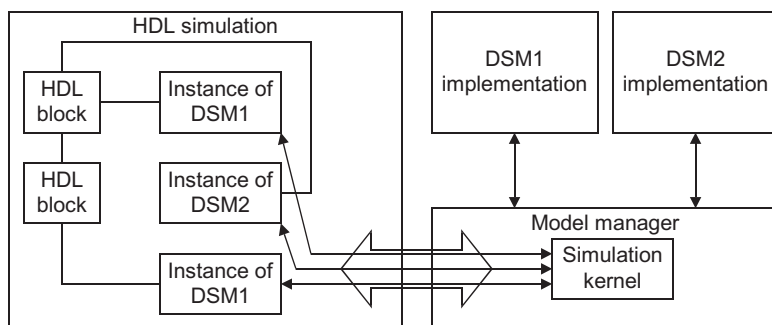


Figure 1-1 DSM integration

Note

For synthesizable cores, the DSM is a pre-implementation, pre-synthesis model. It does not contain any scan test insertion or BIST and is not a SOM.

Chapter 2

Out Of Box Procedures

This chapter describes how to install the DSM for initial use and test the installation. It contains the following sections:

- *Installation* on page 2-2
- *Directory structure and files* on page 2-3
- *Licensing* on page 2-6
- *Install the SWIFT model libraries* on page 2-9
- *Testing your model installation* on page 2-10.

2.1 Installation

The initial product bundle must be located in your current working directory as described in the release note accompanying the deliverables.

The package files include an installation script, `install.csh`. Run this from your working directory with the following command:

```
./install.csh
```

Follow the on-screen instruction to install the model. This includes the acceptance of the license agreement. The install script prompts you for an installation directory. If you choose a directory that does not exist then the script creates it for you. Ensure, if necessary, that you have the appropriate permissions to create this directory.

Confirm that your deliverables conform to those described in *Directory structure and files* on page 2-3.

2.2 Directory structure and files

The installed package contains all of the files that you require to use the model on a UNIX or Linux workstation. It expands into a directory named `simulation_models`. Inside `simulation_models` is a subdirectory named `DSM`, and inside this is another directory named according to the simulator and platform.

Figure 2-1 shows the `simulation_models` directory structure:

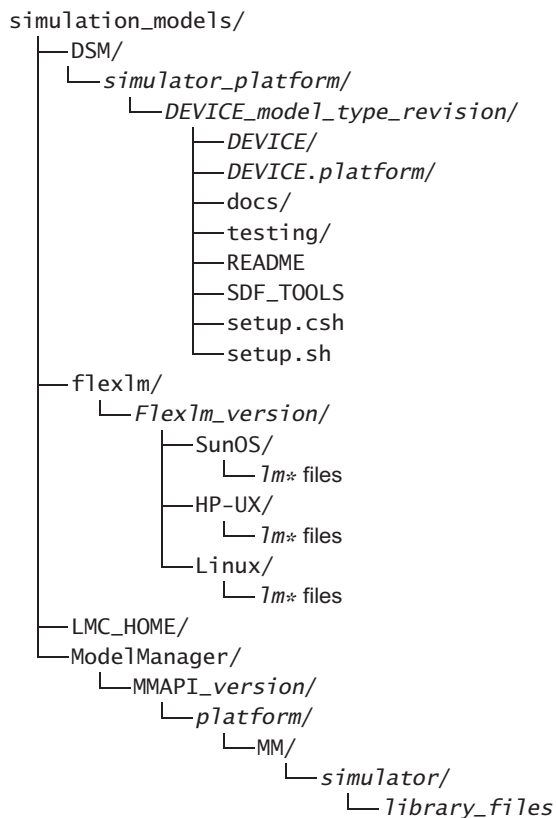


Figure 2-1 Directory structure

Note

- Figure 2-1 shows the `flex1m` directory. This is only applicable to licensed models. See *Licensing* on page 2-6.

- Figure 2-1 on page 2-3 shows the ModelManager directory. See *Configuring the model manager* on page 3-4.

The major directories and files located in *DEVICE_model_type_revision* are:

README	Initial installation instructions.
DEVICE	This directory contains all of the required model files. These include: <i>DEVICE.so</i> The shared object library contains <i>Intellectual Property</i> (IP) relating to the model. If the DSM does not contain any SWIFT components then all of the functional IP related to the model is contained in this library. Note You must not mix parts of the DSM with earlier versions because this leads to the DSM not working correctly. Only use the supplied version of the library.
<i>DEVICE.sdf</i>	An example SDF file for the DSM.
<i>DEVICE.sdft</i>	The SDF template for the DSM.
<i>DEVICE.v</i> or <i>DEVICE.vhd</i>	The HDL SDF timing wrapper for the DSM.
<i>extra_wrapper/DEVICE.v</i> or <i>DEVICE.vhd</i>	The non-timing wrapper for the DSM.
<i>DEVICE.platform</i>	This contains all of the files that you require to install the SWIFT model for a specific platform. This is not present in some DSMs.
testing	This directory contains a <i>Condensed Reference Format</i> (CRF) vector player testbench known as the CRFTESTER. The CRFTESTER reads in a CRF vector file that provides the vectors for the test and also the expected outputs. The CRFTESTER also reads in a file called the CTRM file that provides information about the relative strobe times of the vectors with respect to the simulation cycle. The tester then simulates the vectors on the DSM and checks the outputs with the expected values that are in the CRF file.

docs	This directory contains some documentation related to the DSM: • README_2 • CRF readme file • model manager application notes.
SDF_TOOLS	This directory contains the SDF tools, sdfgen and sdfremap, that are provided with the DSM. These are executables that are compiled for the same platform as the DSM.
setup.(c)sh	Setup files.

2.3 Licensing

This section is only applicable to license-managed DSMs.

License-managed DSMs use Macrovision FLEXlm. The `simulation_models/flexlm` directory contains binaries for the FLEXlm license management software.

You must ensure that the license server is running with the correct license key.

How to obtain and use a license key is described in:

- *Obtaining your license key*
- *License server program*
- *Setting your license environment* on page 2-8.

2.3.1 Obtaining your license key

You might not need a new key if you already have a license key for another DSM of this device. You can check whether your DSM requires a new key by comparing the FEATURE entries in your existing license file with the license feature required by this DSM. Each FEATURE entry in a license file begins with the keyword FEATURE and ends with a hexadecimal number that might be enclosed in quotes. The license feature enabled by a particular FEATURE entry is the second field, which for a DSM usually begins with ModLib_DSM or ModLib_DSM2. The license feature required by this DSM is recorded in the README file.

If your DSM requires a new key then you are sent a serial number, such as WT123-123456789. When you have received this then you can login to the ARM on-line license request system at <https://license.arm.com> and obtain your license key. This web site enables you to:

- generate licenses for your model
- view all licenses for your account
- edit your registration details.

You must have a username and password to login. If you are a new user then you must register to obtain a username and password. On-line instructions and help pages about how to register are available at <https://license.arm.com>.

2.3.2 License server program

You must run the ARM license server program, `armld`, and inform it about the model license after you have received your model license key. This program and other license utilities can be found in the `simulation_models/flexlm/version/platform` directory in your DSM installation, where *platform* is the operating system that you run your license server on. This directory is named `$FLEXLM_BIN` in this description for clarity.

Using the license server program is described in:

- *If you have an existing armlmd running*
- *If you do not have an existing armlmd running on page 2-8.*

If you have an existing armlmd running

You must run version 9.0 or later of the ARM license server for this DSM. If you already have a license server running then find out its version using `lmver` as follows:

`$FLEXLM_BIN/lmver existing_armlmd` where *existing_armlmd* is the path to your existing armlmd program.

You are not required to do anything else to set up your license server if your existing armlmd is version 9.0 or later and your DSM does not require a new key.

If your existing armlmd is earlier than version 9.0 or your DSM requires a new key then proceed as follows:

If your existing armlmd is earlier than version 9.0

1. Check that no ARM tools served from armlmd are running by running `lmstat` on the license server computer:
`$FLEXLM_BIN/lmstat -S armlmd`
2. After you have ensured that the license server is not being used then stop it by running `lmdown` on the license server computer as follows:
`$FLEXLM_BIN/lmdown -c existing_file`
where *existing_file* is the path to your existing ARM license file.
3. Copy the FEATURE entries from your existing license file onto the end of the file containing the new key for your DSM. Do not copy a FEATURE entry if it is for another DSM of the same device. You can tell which device that a DSM FEATURE entry is for by looking at its second field. This usually begins with `ModLib_DSM` or `ModLib_DSM2`.

Note

If your existing license file contains any DSM FEATURE entries that contain the text `SIGN=` rather than `SIGN2=` then you require a new version of that DSM and a new key that is compatible with FLEXlm version 9.0. Contact ARM to obtain these, give details of all the DSMs and ARM license keys that you have.

4. Follow the instructions in *If you do not have an existing armlmd running* on page 2-8.

If your existing armlmd is version 9.0 and you have a new key for this model

1. Copy the FEATURE entry from your new key onto the end of your existing license file. The FEATURE entry begins with the keyword FEATURE and ends with SIGN2= followed by a quoted hexadecimal number.
2. Inform your license server about the new key by running `lmreread` on your license server computer:
`$FLEXLM_BIN/lmreread -c license_file`
where *license_file* is the path name of your ARM license file.

If you do not have an existing armlmd running

You must start armlmd as follows:

1. Copy the `$FLEXLM_BIN/armlmd` program to a permanent location.
2. Ensure that the line beginning with SERVER in your license file gives the correct path to armlmd, including the filename.
3. Copy the ARM license file to a permanent location.
4. You can now use the following command to start armlmd on your license server computer:
`$FLEXLM_BIN/lmgrd -c filename > logfile &`
where *filename* is the path name of the license file and *logfile* is the path to a file where the license server keeps a log of licensing operations.

Note

If you want to start armlmd automatically at boot time then add the command to either `/etc/rc.boot` or `/etc/rc.local`.

2.3.3 Setting your license environment

Before using the model, set the environment variable `ARMLMD_LICENSE_FILE` to the path to your ARM license file that contains the key for the model.

If the model fails to start because it cannot find a license, ensure that there are no other ARM license files specified in either:

- your `LM_LICENSE_FILE` environment variable
- the `.flexlmrc` file in your home directory.

The `.flexlmrc` file is written automatically by FLEXlm, so you might not be aware that it exists.

2.4 Install the SWIFT model libraries

Note

This section is only applicable if your model includes a SWIFT model, that is, the directory structure contains a *DEVICE.platform* directory.

Follow these instructions if either of the following are applicable:

- you did not run `install.csh` successfully
- you want to install the SWIFT model again.

Note

You are only required to install the SWIFT model libraries once.

Run the install script to install the SWIFT model in the default area pointed to by the `LMC_HOME` environment variable:

```
$DEVICE_HOME/DEVICE.platform/install_swift.csh
```

If you plan to run two or more different SWIFT models in the simulation then you must install all of them into the same `LMC_HOME`.

Ensure that you set `LMC_HOME` to the same location before installing each SWIFT model, then run the install script:

```
$DEVICE_HOME/DEVICE.platform/install_swift.csh
```

If the model installs correctly then the script ends with the message:

Install Complete

2.5 Testing your model installation

You must set up your model before you can test the installation. See *Setting up your model* on page 3-2.

Testing your model installation is described in:

- *Setting up your simulator environment*
- *Running the test script.*

2.5.1 Setting up your simulator environment

See your simulator documentation to do this.

2.5.2 Running the test script

Run the relevant test script:

With SDF timing `$DEVICE_HOME/testing/crftest.csh`

With no timing `$DEVICE_HOME/testing/crftest_extra.csh`

If there are no errors then the crftester simulation runs the vector set with zero vectors failed and the following completion message is displayed:

The *DEVICE* DSM is correctly setup

An example of crftester simulation log trace is supplied:

With SDF timing `$DEVICE_HOME/testing/reference_crftest.log`

With no timing `$DEVICE_HOME/testing/reference_crftest_extra.log`

Chapter 3

Configuring a Model

This chapter describes how to configure the DSM and associated components for use. It contains the following sections:

- *Setting up your model* on page 3-2
- *Configuring the HDL wrapper* on page 3-3
- *Configuring the model manager* on page 3-4
- *Configuring the DSM* on page 3-10.

3.1 Setting up your model

Setting up your model is described in:

- *Set the `DEVICE_HOME` environment variable*
- *Set up the model environment*

3.1.1 Set the `DEVICE_HOME` environment variable

Go to the model directory:

```
cd ../simulation_models/DSM/simulator_platform/DEVICE_model_type_revision
```

Set the environment variable:

```
C shell      setenv DEVICE_HOME `pwd`  
  
Bourne shell DEVICE_HOME=`pwd`  
              export DEVICE_HOME
```

3.1.2 Set up the model environment

Source the setup script:

```
C shell      source $DEVICE_HOME/setup.csh  
  
Bourne shell . $DEVICE_HOME/setup.sh
```

Table 3-1 lists the environment variables that this configures.

Table 3-1 Model environment variables

Variable	Function	Default value
<code>DIR_DEVICE</code>	Points to the directory that contains the shared library, <code>DEVICE.so</code>	<code>\$DEVICE_HOME/DEVICE</code>
<code>MG_LIB</code>	Points to the directory that contains the model manager	<code>simulation_models/ModelManager/MM_Version/platform/MM</code>
<code>LMC_HOME</code>	Points to the SWIFT model installation area	<code>simulation_models/LMC_HOME</code>
<code>LD_LIBRARY_PATH</code> or <code>SHLIB_PATH</code>	Path to find the model manager ^a	<code>\${MG_LIB}/simulator:\${LD_LIBRARY_PATH}</code> or <code>\${MG_LIB}/simulator:\${SHLIB_PATH}</code>

a. The `LD_LIBRARY_PATH` or `SHLIB_PATH` environment variable is only set up for Synopsys VCS models. Other simulators do not require it.

3.2 Configuring the HDL wrapper

A model can have an SDF-annotatable timing shell associated with it or you can simulate it without timing. Select the appropriate HDL wrapper for the kind of simulation that you want.

Note

The DSM with its timing shell is useful for:

- timing sign-off with a non-synthesizable processor
- some preliminary timing simulation with a synthesizable processor.

The DSM is not suitable for timing sign-off of a synthesizable processor. You must use a dedicated SOM instead.

The HDL wrapper with timing shell is located in `$DEVICE_HOME/DEVICE` and the HDL wrapper without the timing shell is located in `$DEVICE_HOME/DEVICE/extra_wrapper`.

The wrapper file includes fragments of HDL that are necessary to instantiate the DSM.

Note

Note the following:

- Do not modify the HDL wrapper because this might result in the model failing in a subtle fashion. ARM Limited cannot support your DSM if the HDL wrapper is modified.
 - You must not mix parts of the DSM with earlier versions because this leads to the DSM working incorrectly. Only use the HDL wrapper file that is supplied with the current version of the DSM.
-

3.3 Configuring the model manager

Configuring the model manager is described in:

- *About the model manager*
- *Configuring your simulator to use the model manager* on page 3-5
- *How the model manager loads a model* on page 3-9.

3.3.1 About the model manager

The DSM is termed a foreign-language model because it is written in a language that is not the native HDL of the simulator. Different simulators can have different interfaces for communicating with foreign-language models. The purpose of the model manager is to provide a simulator-independent standard interface to the models that it manages and to handle all communication between those models and the simulator.

The model manager is designed so that you do not have to load each model separately into the simulation. Instead, the simulator opens or links against a single copy of the model manager that handles all of the requests to load DSMs in that simulation. A single model manager can service multiple instances of multiple models in a simulation.

The instructions on using the model manager vary between simulators because different simulators have different methods of integrating foreign-language models into their simulations.

The model manager is platform and simulator dependent but is independent of which DSM you are using. It is shipped with each DSM. You must always use the latest version of model manager if you have more than one DSM in your design. The environment variable, `MG_LIB`, points to the root directory of the model manager as shown in Figure 2-1 on page 2-3 and described in *Set up the model environment* on page 3-2.

————— **Note** —————

If you have more than one DSM in a simulation and one of the DSMs is older than the other then you must ensure that you are using the most recent model manager version or your simulation will not start. The most recent shared object library only loads with the most recent model manager version.

3.3.2 Configuring your simulator to use the model manager

This section describes how to configure your simulator to use the model manager with:

- *Cadence NC-Verilog*
- *Cadence NC-VHDL* on page 3-6
- *Cadence Verilog-XL* on page 3-6
- *MTI ModelSim Verilog* on page 3-7
- *MTI Modelsim VHDL* on page 3-7
- *Synopsys VCS* on page 3-7.

Note

With the exception of Cadence Verilog-XL, note the following when using the model manager:

If you want use the 64-bit version of your simulator then you must ensure that you have the 64-bit variant of the DSM. The platform variant supported by the DSM is documented at the beginning of the README file, for example, a DSM that supports the 64-bit simulator variant under Solaris, has the following text:

Platform : SunOS-64

If the platform variant of your DSM is shown as SunOS then the model is intended to be used with the 32-bit version of your simulator only. The 32-bit version of the DSM does not run under the 64-bit version of the simulator.

Cadence NC-Verilog

NC-Verilog uses the Verilog PLI to load foreign-language models.

Use the `-loadpli1` command line option when you run `ncelab` to include the model manager in your simulation. This loads the model manager dynamically. The name of the model manager library is:

Solaris and Linux `${MG_LIB}/cadence_nc_verilog/mm_nc_dynamic.so`

HP-UX `${MG_LIB}/cadence_nc_verilog/mm_nc_dynamic.sl`

The entry point function is `mgboot_nc`.

For example, if your simulation contains one or more DSMs with a top-level Verilog module named `testtop` then you can elaborate the simulation after compiling the Verilog, by using the command:

```
ncelab -loadpli1 \
      ${MG_LIB}/cadence_nc_verilog/mm_nc_dynamic:mgboot_nc \
```

testtop

You do not have to rebuild libpli.so or any other library.

Note

You must use the -nbasync option to ncsim when you use NC-Verilog.

Cadence NC-VHDL

NC-VHDL uses the foreign module integration interface to load foreign-language models.

Use the -loadfmi command line option when you run ncsim to include the model manager in your simulation. This loads the model manager dynamically. The name of the model manager library is:

Solaris and Linux \${MG_LIB}/cadence_nc_vhdl/mm_ncvhdl_dynamic.so

HP-UX \${MG_LIB}/cadence_nc_vhdl/mm_ncvhdl_dynamic.sl

The entry point function is mgboot_ncvhdl.

For example, if your simulation contains one or more DSMs with a top-level VHDL configuration named testtop then you can run the simulation after compiling the VHDL, by using the command:

```
ncsim -loadfmi \
      ${MG_LIB}/cadence_nc_vhdl/mm_ncvhdl_dynamic:mgboot_ncvhdl \
      testtop
```

You do not have to rebuild libfmi.so or any other library.

Cadence Verilog-XL

Verilog-XL uses the Verilog PLI to load foreign-language models.

Use the +loadpli1 command line option when you run verilog to include the model manager in your simulation. This loads the model manager dynamically. The name of the model manager library is:

Solaris and Linux \${MG_LIB}/cadence_xl_verilog/mm_xl_dynamic.so

HP-UX \${MG_LIB}/cadence_xl_verilog/mm_xl_dynamic.sl

The entry point function is mgboot_xl.

For example, if your simulation contains one or more DSMs and Verilog files named `my_model.v` and `my_testbench.v` then you can start the simulation by using the command:

```
verilog \
+loadpli=${MG_LIB}/cadence_xl_verilog/mm_xl_dynamic:mgboot_xl \
my_model.v my_testbench.v
```

You do not have to rebuild `libpli.so` or any other library.

MTI ModelSim Verilog

ModelSim Verilog uses the Verilog PLI to load foreign-language models.

ModelSim uses the `Veriuser` entry from the `modelsim.ini` file to load PLI models. This entry is a list of shared libraries to load. It must be located in the `[vsim]` section of `modelsim.ini`. The `$MG_LIB/mti_modelsim_verilog/libmgmm.so` library must appear in the list to include the model manager in your simulation. For example:

```
[vsim]
Veriuser = $MG_LIB/mti_modelsim_verilog/libmgmm.so
```

The file name extension for the `libmgmm` library is always `.so` irrespective of the platform you are working with.

Do not add the shared libraries for the DSMs, such as `device.so`, to the list.

If you must simulate with other PLI applications then add the shared libraries containing those applications to the list.

MTI Modelsim VHDL

The model VHDL wrapper causes the simulator to automatically load the model manager through the ModelSim FLI. You only have to compile the VHDL wrapper.

————— Note —————

If you are compiling with a VHDL/SDF wrapper then use:

```
vcom -novitalcheck
```

The wrapper compiles incorrectly if you do not use this.

Synopsys VCS

VCS uses the Verilog PLI to load foreign-language models. You must make VCS aware of your DSMs by creating a new `simv` simulator executable that is linked against the model manager. You can use the `vcs` command to do this.

Specify the following on the vcs command line to build an executable containing a model manager:

- A PLI table for the model. For a single model or a model with a CRF tester, you can use the supplied `pli.tab`, by including the following on the command line:

`-P pli.tab`

The example `pli.tab` is located in `$DEVICE_HOME/DEVICE`:

```
$mm call=mm_call misc=mm_misc
acc+=rw,cbk:DEVICE+
```

- The location and name of the model manager library. Include the following on the command line:

Solaris `-LDFLAGS "-L${MG_LIB}/synopsys_vcs_verilog" -lmgmm`

Linux `-LDFLAGS "-L${MG_LIB}/synopsys_vcs_verilog" -lmgmm -ldl`

HP-UX `-LDFLAGS \`
`"-L${MG_LIB}/synopsys_vcs_verilog -Wl,-a,shared_archive,-Fz" \`
`-lmgmm -ldld`

The extra linking flags used on Linux and HP-UX are necessary to enable VCS to interact with shared libraries in the appropriate way.

- The Verilog files necessary for the simulation, including the Verilog wrapper for the model.

For example, consider a model named `device.v` with a Verilog testbench named `device_test.v`. Both the testbench and the model Verilog wrapper are in the current directory. To build an executable on Solaris, change to the directory containing the model and run the command:

```
vcs -Mupdate -P pli.tab \
-LDFLAGS "-L${MG_LIB}/synopsys_vcs_verilog" \
-lmgmm device.v device_test.v
```

Alternatively, to build an executable on HP-UX, use:

```
vcs -Mupdate -P pli.tab -LDFLAGS \
"-L${MG_LIB}/synopsys_vcs_verilog -Wl,-a,shared_archive,-Fz" \
-lmgmm -ldld device.v device_test.v
```

Before running the simulation, you must also ensure that for:

Solaris and Linux The `${MG_LIB}/synopsys_vcs_verilog` directory appears in the `LD_LIBRARY_PATH` environment variable;

HP-UX The `${MG_LIB}/synopsys_vcs_verilog` directory appears in the `SHLIB_PATH` environment variable.

Note

In both cases, you must ensure that the `${MG_LIB}/synopsys_vcs_verilog` directory appears in the environment variable *before* the simulator libraries.

To simulate, run the `simv` executable that is created in the current directory.

Adding more PLI code

If you must simulate with other PLI applications then you must alter the PLI table, `pli.tab`, to include the entry points for those applications before building the executable. You must also include any necessary libraries or object files on the `vcs` command line. See the documentation from your simulator vendor for more information about how to do this.

3.3.3 How the model manager loads a model

The model manager loads the shared library for the relevant DSM. This has the same name as the device and the appropriate OS-specific shared library filename extension.

The model manager first looks for the library in the directory specified in the `MG_OBJECT` generic or parameter in the HDL wrapper for the model. This generic is set to the environment variable `DIR_DEVICE` by default, for example `DIR_ARM7TDMI` for an ARM7TDMI DSM.

You can change the directory specified in `MG_OBJECT` when you instantiate the DSM by overriding its value with either a:

- generic in VHDL
- parameter in Verilog.

If the model manager cannot find the specified directory for the library then it looks in the current working directory.

3.4 Configuring the DSM

ARM Limited provides some configurable DSMs such as those for the ARM926EJ-S and ARM1136JF-S processors. The README_2 file in the docs directory of each DSM contains a description of this configurability.

The configurability normally includes:

- the DSM identifier, for multi processor designs.
- the size of the internal memories:
 - instruction cache and TCM
 - data cache and TCM.

Some DSMs might also include:

- JTAG ID for the ARM7TDMI DSM
- gated Clock control for the ARM926EJ-S DSM
- power mode and BIST status for the ARM946ES DSM.

Note

Do not modify the HDL wrapper because this might result in the model failing in a subtle fashion. ARM Limited cannot support your DSM if the HDL wrapper is modified.

You can use Verilog parameters or VHDL generics to configure a DSM. To do this, you must modify the component instantiation of the DSM by adding the lines of the required Verilog defparam or VHDL generic mappings. For example, to configure the ARM926EJ-S DSM:

Verilog Add the following to the instance of the ARM926EJ-S DSM:

```
defparam theARM926EJS.ID           = 5           ;
defparam theARM926EJS.ICACHE_SIZE = 16384       ; // size in bytes
defparam theARM926EJS.DCACHE_SIZE = 32768       ; // size in bytes
defparam theARM926EJS.DisableBkClkGate = 0 ; // gated clock not disabled
// COMPONENT INSTANTIATION
ARM926EJS theARM926EJS (HADDR, ...)
```

Note

... indicates more line data that is not relevant to the subject.

VHDL Add the following to the instance of the ARM926EJ-S DSM:

```
-- COMPONENT DECLARATION
component ARM926EJS
  Generic(ID : INTEGER;           -- DEVICE IDENTIFIER
    ICACHE_SIZE : INTEGER;       -- ICACHE SIZE
```

```

        DCACHE_SIZE : INTEGER;          -- DCACHE SIZE
        DisableBlkClkGate : INTEGER; -- control of gated clock

Port( HADDR : out std_logic_vector(31 downto 0);
...

        -- COMPONENT INSTANTIATION
the ARM926EJS : ARM926EJS
    Generic map (
        ID          => 5,                -- DEVICE IDENTIFIER ( 0,1,...7)
        ICACHE_SIZE  => 16384,            -- ICACHE SIZE in bytes
        DCACHE_SIZE  => 32768,            -- DCACHE SIZE in bytes
        DisableBlkClkGate => 0,          -- gated clock not disabled
    )
port map ( HADDR, ...)

```


Chapter 4

Using a Model

This chapter describes timing issues for synthesizable and non-synthesizable cores, the top-level registers, and how to control instruction stream tracing. It contains the following sections:

- *Facilities for debugging a simulation* on page 4-2
- *Limitations of use* on page 4-6.

4.1 Facilities for debugging a simulation

The DSM debugging facilities are described in:

- *Programmer's model*
- *Executed instruction stream tracing* on page 4-3

4.1.1 Programmer's model

The model exports certain aspects of the programmer's model as signals that are visible at the top-level of the HDL wrapper. Table 4-1 lists these.

Table 4-1 Top-level registers

Register	Description
r0-r14	User and System mode r0-r14.
pc	Pseudo Program Counter (PC).
r13irq-r14irq	IRQ Mode r13-r14.
r13svc-r14svc	Supervisor Mode r13-r14.
r13abt-r14svc	Abort Mode r13-r14.
r13und-r14und	Undefined Mode r13-r14.
r8fiq-r14fiq	FIQ Mode r8-r14.
cpsr	<i>Current Program Status Register (CPSR).</i>
spsrfiq	FIQ Mode <i>Saved Program Status Register (SPSR).</i>
spsrirq	IRQ Mode SPSR.
spsrsvc	Supervisor Mode SPSR.
spsrabt	Abort Mode SPSR.
spsrund	Undefined MODE SPSR.

Figure 4-1 on page 4-3 shows the bit assignments for *Program Status Registers (PSRs)*. These registers:

- hold information about the most recently performed ALU operation
- control the enabling and disabling of interrupts
- set the processor operating mode.

The model also exports the clock cycle count, `ClkCount`, for debug. This is correlated with the clock cycle count in the `log.eis` disassembler output file.

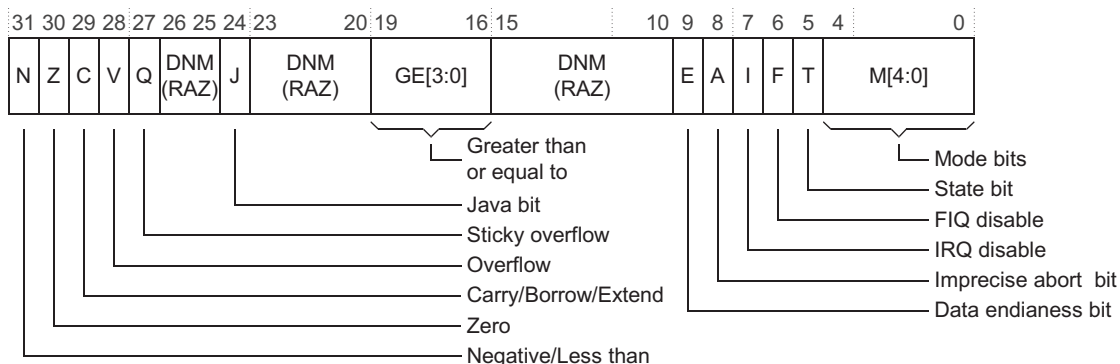


Figure 4-1 Program status register bit assignments

Note

Not all processors implement all of the Program Status Register fields shown in Figure 4-1.

4.1.2 Executed instruction stream tracing

Most of the DSMs provide a built-in disassembler that generates an output `log.eis` file for EIS tracing. See Appendix A *EIS Output Format* for more information. The availability of the disassembler and its on and off control vary between cores. The `README_2` file in each DSM package contains more information. A summary of these is provided in:

- *Synthesizable ARM7 and ARM9 cores:*
- *ARM1020E and ARM1022E cores* on page 4-4
- *ARM1026EJ-S core* on page 4-5
- *ARM11 cores* on page 4-5.

Synthesizable ARM7 and ARM9 cores:

By default, EIS tracing is on and is logged to `log.eis` in the simulation running directory. There is no output to the screen.

To turn the EIS tracing off:

C shell `setenv eis_dont_generate "any string"`

Bourne shell `eis_dont_generate= "any string"`
 `export eis_dont_generate`

To turn echo to stdout on:

C shell `setenv eis_echo_to_stdout "any string"`

Bourne shell `eis_echo_to_stdout= "any string"`
 `export eis_echo_to_stdout`

ARM1020E and ARM1022E cores

By default, EIS tracing is off.

To turn EIS tracing on:

C shell `setenv DEVICE_DISASS_STDOUT "any string"`

 or

 `setenv DEVICE_DISASS_TO_FILE`

Bourne shell `DEVICE_DISASS_STDOUT= "any string"`
 `export DEVICE_DISASS_STDOUT`

 or

 `DEVICE_DISASS_TO_FILE= "any string"`
 `export DEVICE_DISASS_TO_FILE`

To turn EIS off:

C shell `unsetenv DEVICE_DISASS_STDOUT`

 or

 `unsetenv DEVICE_DISASS_TO_FILE`

Bourne shell `DEVICE_DISASS_STDOUT=`
 `export DEVICE_DISASS_STDOUT`

 or

 `DEVICE_DISASS_TO_FILE=`
 `export DEVICE_DISASS_TO_FILE`

Note

DEVICE is ARM1020E or ARM1022E

ARM1026EJ-S core

By default EIS tracing is off with no displayed echo.

To turn the EIS tracing on:

C shell `setenv eis_trace_on "any string"`

Bourne shell `eis_trace_on= "any string"`
 `export eis_trace_on`

To turn echo to stdout on:

C shell `setenv eis_echo_to_stdout "any string"`

Bourne shell `eis_echo_to_stdout= "any string"`
 `export eis_echo_to_stdout`

ARM11 cores

By default EIS tracing is off. When it is on then EIS is logged in a file named `tarmac.log`. There is no displayed echo.

To turn the EIS tracing on:

C shell `setenv tarmac_on "any string"`

Bourne shell `tarmac_on= "any string"`
 `export tarmac_on`

To turn echo to stdout on:

C shell `setenv tarmac_echo_to_stdout "any string"`

Bourne shell `tarmac_echo_to_stdout= "any string"`
 `export tarmac_echo_to_stdout`

See the *ADS Debug Target Guide* for more information about the `tarmac` file format.

4.2 Limitations of use

Although DSMs match the architecture and functionality of the appropriate core designs, they are subject to the limitations described in:

- *Unsupported simulator functions*
- *Internal scan chain modeling*
- *Caches and registers*
- *Timing model.*

4.2.1 Unsupported simulator functions

The following simulator functions are not supported:

Restart Return the simulation back to time zero without terminating the simulation.

Save and Restore, also known as checkpointing

Save the simulation at a determined point of time, also known as a snapshot, and restore the simulation to that point of time.

4.2.2 Internal scan chain modeling

DSMs are derived from the RTL description of the core that they model. The final netlist for the core might contain internal scan chains that were added during synthesis. It is not possible to use DSMs to model these scan chains because they do not exist in the device RTL. However, the scan chains are modeled by the SOM of a device.

4.2.3 Caches and registers

Although it is possible to view the register values contained in the DSM simulation, it is not possible for you to introduce any test data directly into the caches or registers, because this cannot be performed in the RTL from which the DSM is derived.

4.2.4 Timing model

Limitations of the timing model on page 5-26 describes these.

Chapter 5

Timing

This chapter describes timing issues and some of the facilities provided to aid the annotation process. It contains the following sections:

- *About the timing shell* on page 5-2
- *Multiple timing paths* on page 5-8
- *SDF annotation* on page 5-10
- *Interconnect delays* on page 5-18
- *Use of negative timing checks* on page 5-20
- *STA and HDL annotated simulation differences* on page 5-23
- *Limitations of the timing model* on page 5-26.

Note

This chapter is applicable for:

- timing sign-off with a non-synthesizable processor
- some preliminary timing simulation with a synthesizable processor.

This chapter is not applicable if you want to do timing sign-off for a synthesizable processor because the DSM is not suitable for this. You must use a dedicated SOM instead.

5.1 About the timing shell

From the point of view of the simulator, the DSM is a large single component in the simulation. The model is sensitive to all of its inputs and can, as far as the simulator is aware, drive output values as a response to any one of them changing. It is treated by the simulator as a large combinatorial block. The model and simulator provide no special treatment to clock signals. They are considered as input signals, like any other, that can cause output events to be generated.

The DSM does not include any information about the timing behavior of the core but it does provide a timing wrapper for you to backannotate your own timing information onto. You can backannotate timing information onto a DSM from an SDF file, as described in IEEE 1497-2001, using the annotation facilities provided by the simulator that you are using.

You can have difficulty in successfully integrating DSMs into a timing flow because they are large single components, and there are consequential implications for the level of timing accuracy that you can achieve. This chapter describes the way that DSM timing works, some limitations of the DSM timing model, and some suggestions on how to make a DSM best fit with your timing flow.

The timing shell is described further in:

- *The SDF HDL wrapper*
- *The pin-to-pin timing model* on page 5-3.

5.1.1 The SDF HDL wrapper

When you backannotate timing information from an SDF file, the simulator incorporates the timing information into the simulation by attaching it to placeholders in the DSM HDL wrapper. Figure 5-1 on page 5-3 shows the structure of the HDL wrapper.

The wrapper has several levels of hierarchy. All these levels exist in the provided single HDL timing wrapper file.

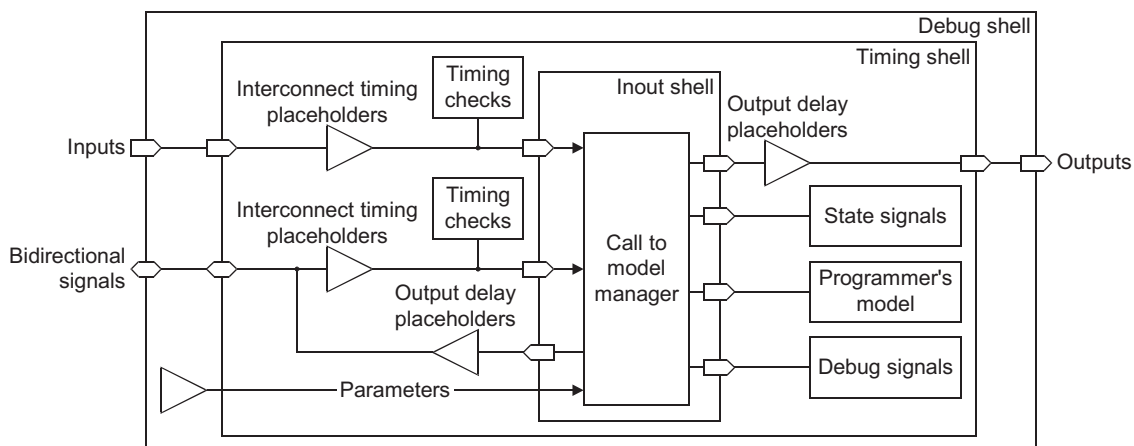


Figure 5-1 HDL wrapper structure

5.1.2 The pin-to-pin timing model

The following scenarios illustrate the behavior of a timing shell:

- Zero delay model core* on page 5-4

Initially, the behavior of a simple synchronous block with no timing functionality is described.
- Addition of timing shell* on page 5-4

The model has an external pin-to-pin timing shell added around it, implementing setup and hold checks, in addition to a delay on the output. It demonstrates that the addition of the timing shell does not affect the functionality of the model.
- Negative timing constraints* on page 5-7

The implications of negative timing constraints, for example, negative setup and hold times are examined. It shows you that a simple pin-to-pin timing shell, used in conjunction with a zero-delay core, cannot be used to model negative timing constraints. To support negative timing constraints, a modification to the basic implementation is described in *Use of negative timing checks* on page 5-20.

Zero delay model core

The first example is a simple synchronous block with:

- a clock, **CLK**
- an input signal **IN**, that is sampled on the rising edge of the clock
- an output **OUT**, that is driven on the falling edge of the clock.

Figure 5-2 shows a simple synchronous block with clock.

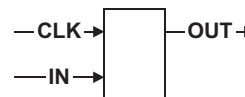


Figure 5-2 Simple synchronous block with clock

The sequence of events from the point of view of the model, without a timing shell is:

1. All signals are set to zero.
2. At 10ps, the **IN** input rises to 1. Because the block is sensitive to changes on **IN**, it is evaluated but the behavior of the block is so that no action is taken.
3. At 20ps, **CLK**, rises to 1. The model is evaluated and latches the value of **IN** internally.
4. At 25ps, **IN** falls to 0. The model is evaluated, but takes no action.
5. At 30ps, **CLK** falls to 0. The model is evaluated and determines that a drive to **OUT** is required.
6. Still at 30ps, but in a subsequent simulation evaluation, the **OUT** output, rises to 1 because of the drive from the block model.

This sequence of events is for a zero-delay, behavioral model, a DSM, with no timing shell. The model is sensitive to all of its inputs and drives its outputs immediately. Because of the simulation semantics of a DSM, no other part of the simulation is evaluated in parallel with the DSM evaluation. Consequently other functional blocks in a simulation cannot insert values between individual DSM evaluation steps even if they are set to drive its inputs at the same simulation time.

Addition of timing shell

In this case a pin-to-pin timing shell is placed around the periphery of the model. The timing shell, like the model is also behavioral code. It monitors input events without affecting their passage, this is only true in the simple case, see *Use of negative timing checks* on page 5-20 and *Interconnect delays* on page 5-18 for more information, and inserts a delay-buffer through which output events propagate. Assume that the timing shell is annotated from the following SDF fragment:

```
(SETUP (POSEDGE CLK) IN (5))
(HOLD (POSEDGE CLK) IN (6))
(IOPATH (NEGEDGE CLK) OUT (3))
```

The timing shell has no special information about the behavior of the device that it is wrapped around. It also does not have any special knowledge of what the signals are used for. For example, clocks versus synchronously sampled inputs.

The sequence of events and their consequences is modified as follows:

1. Initially all signals are set to zero.
2. At 10ps, **IN** rises to 1. The timing shell notes the current simulation time 10ps as the time when **IN** changes and passes the change through to the underlying model. This behaves as described in *Zero delay model core* on page 5-4 and no action is taken.
3. At 20ps the clock, **CLK**, rises to 1. The timing shell notes the current simulation time 20ps as the time when the clock rose. A setup check is now performed of **IN** against the positive edge of the clock by subtracting the stored time when **IN** changed 10ps from the current simulation time, 20ps. This value 10ps is greater than the setup time of 5ps so no warning is emitted. The timing shell passes the clock change event through to the underlying model which latches the value of **IN** as before.
4. At 25ps, **IN** falls to 0. The timing shell notes the current simulation time when **IN** changed and replaces its previous value, 10ps. It then performs a hold check by subtracting the stored time when the clock rose 20ps from the current simulation time 25ps, and notes that the actual hold time of **IN** against the positive edge of the clock was 5ps. This is less than the 6ps indicated in the SDF file, and so a hold violation message is printed in the simulation log. The event on **IN** is passed through to the model as before and the hold violation has no effect on the underlying model behavior. The output is not forced to X because of the timing violation.
5. At 30ps, **CLK** falls to 0. The timing shell notes this time as the time when the clock fell. No more action is necessary. The model is evaluated and determines that a drive to **OUT** is required.
6. Still at 30ps, but in a subsequent simulation evaluation, the output **OUT**, rises to 1 because of the drive from the block model. The timing shell intercepts this drive from the model and notes that the negative edge of the clock happened at the current simulation time, even though this occurred from a previous simulation evaluation. It therefore inserts a delay on the propagation of **OUT** to the outer edge of the timing shell of 3ps.
7. At 33ps, the delayed version of **OUT** propagates from the outer edge of the timing shell, and into the wider simulation, see Figure 5-3 on page 5-6.

Note

The model core drives this output at 30ps simulation time, the subsequent delay is implemented entirely by the timing shell.

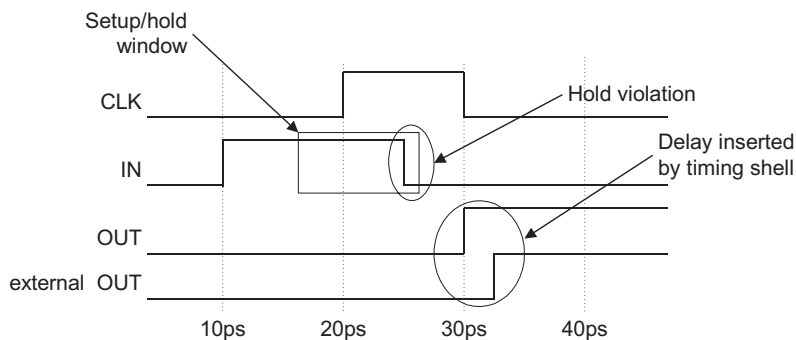


Figure 5-3 A modified sequence of events

As an event simulation, the behavior differs from physical hardware in a number of ways. Firstly, if we examine the point where the hold-window for the sampling of **IN** by the device was violated:

- In physical hardware, this renders the subsequent behavior of the device unpredictable because it cannot be guaranteed that the correct value of **IN** was actually sampled and used in the evaluation of the output.
- In the event simulation, the sampling of synchronous inputs occurs as a discrete event at the clock edge. The resultant hold violation is a cosmetic warning that does not affect the subsequent behavior of the simulation, and must be taken to indicate that the simulation can no longer be assumed to match reality after this point.

Note

ARM DSMs do not drive outputs to X in response to a timing violation.

Secondly, in this example, the model evaluates its outputs when it has all pertinent information. This is when it receives the negative-edge event on the clock.

- In physical hardware, the delay of 3ps on the output **OUT**, is partly because of internal path delays in the device and partly because of the load to be driven by the output.

- In this event simulation, the new value is calculated immediately and the timing shell simulates the cumulative delays after the fact. This works for the purposes of simulation provided that the output value can really be calculated at the point when the clock edge that drives them occurs. This might not always be the case, an explanation is provided in the next section.

Negative timing constraints

In this example there is a high performance device where a sampled input has a negative-setup time, and the output it affects is driven by the same clock edge as when the input is sampled. Figure 5-4 shows this situation.

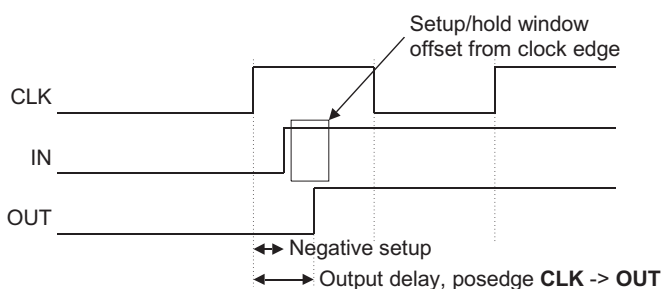


Figure 5-4 High performance device with negative setup time

In this situation, the value of **IN** can change for a certain time after the clock edge, and still affect the value of the derived output. For example, this can be because of a delay in propagating the clock around the inside of a large device. In reality the output **OUT** is driven at a point some time after the input clock edge is nominally said to have occurred because of propagation delays inside the device. A DSM-style model does not model these delays. The output is driven immediately and delayed by the timing shell. A subsequent change on **IN**, even if it occurs before the final drive of **OUT** by the timing shell, cannot affect the simulation because the model has already driven the result and cannot revoke it.

Simulating this type of behavior in a pin-to-pin timing shell is therefore quite complicated, and if applied to the scenario outlined does not work. See *Use of negative timing checks* on page 5-20 for information on how the timing shell behavior is modified by the simulator to handle these situations.

5.2 Multiple timing paths

In *Addition of timing shell* on page 5-4 it is assumed that the timing shell determines that the positive edge of the clock was the causal event for the output delay. The timing shell cannot work this out for itself because it has no understanding of the logic and state inside the model. In that example, this is not significant because there is only a single possible originating input event that can cause the output to change.

In larger devices it is often the case that an output can change because of multiple causal input events. Examples include multiple clocks such as a memory clock and a test clock, or an asynchronous reset signal. When ARM Limited constructs the timing shell there is a list of possible causal events for each output. If more than one possible causal event occurs at the same simulation time then the timing shell can potentially pick the wrong event because it has no understanding of the actual behavior of the model and therefore no way of knowing what event was causal. Inaccuracies can be avoided in one of two ways:

- ARM Limited can construct the timing shell so that additional information, exported by the model can be used to enable only certain timing arcs dynamically depending on the internal state of the model. These timing arcs are enabled and disabled through the use of the conditional timing arc facility, COND construct, present in SDF.
- The timing constraints on the model might be arranged so that it is not valid for the input events in question to occur at the same time. This is enforced through setup and hold checks.

A special case is described in *State-dependent timing*.

5.2.1 State-dependent timing

A special case involving multiple timing arcs that can be problematic is when two or more timing arcs for a given output delay, that is, an SDF IOPATH construct as described in *SDF annotation* on page 5-10, have the same causal event specified, for example negedge **CLK**, and differ only through their use of the SDF COND facility. These are referred to as internal state-dependent delays and present a particular problem for SDF timing flows. Unless the vendor software generating the SDF data to be annotated can differentiate between the multiple arcs, then a single set of delay times are annotated onto all of them. This results in the delays being the same regardless of the state.

Static Timing Analysis (STA) cannot normally determine what state a device being modeled is in when calculating delay values. Certain devices such as the ARM7TDMI processor exhibit state-dependent timing. STA based timing flows can be used with such a device, but state-dependency is not modeled in these simulations. An alternative

flow that is able to model state-dependent timing behavior is supported, but a description of this flow is beyond the scope of this document. See *ARM Design Signoff Models: Timing Annotation* for more details.

Note

The use of the SDF COND facility does not indicate that state-dependent timing is in effect. True state dependency arises when the use of the COND facility is all that distinguishes otherwise identical timing arcs. COND is also used by the timing shell to disable individual timing arcs when they are not appropriate.

5.3 SDF annotation

VHDL and Verilog logic simulators perform SDF annotation in similar ways, although there are differences in the specific details between the two languages.

Figure 5-5 shows the structure of a simple timing shell.

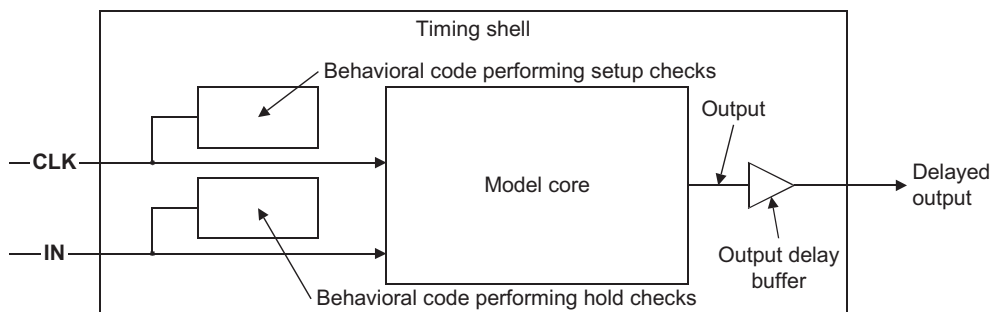


Figure 5-5 Simple timing shell structure

A timing shell is a block of behavioral VHDL or Verilog code although most of the actual behavior is inferred in Verilog implementations where timing constructs are provided as language features. This code is organized into a routine for each signal in the design that is invoked when an event occurs on that signal.

For input signals, the code checks that an event is happening at a permitted time, that is, it is not violating a timing check associated with that signal. The passage of the input signal is not impeded by timing check code, it is only monitored. However, input signals can be delayed for other reasons, See *Interconnect delays* on page 5-18 and *Use of negative timing checks* on page 5-20 for more information.

Output signals are responded to by a buffer that inserts a delay in the propagation of the event into the outside world.

In both of these cases, the timing shell requires time values to work with. Input checks require the setup time and hold time to compare against events. Output delays have to know how long to delay the drive. Unlike normal HDL code, these values are not present in the code itself and instead a place-holder value is used for each timing arc that the timing shell is interested in. These placeholders are set to some nominal value in the HDL source, typically zero, and filled in automatically at the start of the simulation from an SDF file.

When inserting values from an SDF file into a timing shell, the simulator has to know that placeholders must be used to hold the values. It does this by using a standard mapping so that a timing arc in an SDF file has a defined equivalent in the Verilog or VHDL source. In VHDL this is done by mapping timing arcs to the names of VHDL

generics, as described in IEEE 1076.4, where the name encodes the signals and edges involved in the timing arc in addition to any conditions. The Verilog language is slightly different in that the language contains special system tasks for timing arcs that include edge and conditional information in their specification. The SDF annotator in the simulator overrides the default values specified in the source code of any timing arcs that match the arcs that it finds in an SDF file.

Potential difficulties can arise in SDF annotation when there is a mismatch between what is present in an SDF file and the placeholders in the timing shell. Because an SDF file comes from a source that is usually not under the control of the supplier of the timing shell then the potential for mismatches exists. SDF is sufficiently syntactically rich that a timing arc that obviously matches a particular construct in the timing shell might not be found as a match by the SDF annotator. For example, assuming that the timing shell is constructed so that there is a setup check between a signal, **IN**, and the positive edge of a clock, **CLK**, then the following SDF code might be expected:

```
(SETUP (POSEDGE CLK) IN ...)
```

If, however, the SDF file contains a single setup check between **CLK** and **IN**:

```
(SETUP CLK IN ...)
```

A setup check on a nonspecified edge of a clock is a superset of a setup check on a positive edge, and so is theoretically a valid construct to annotate onto the appropriate check in the timing shell. The OVI SDF2.1 standard suggests that, in this situation, annotation must proceed. However, in practice, different simulators handle this situation differently. Verilog simulators generally annotate a specific timing arc from a less specific SDF construct like this. VHDL, however, is less flexible and requires an exact match. Also, if the SDF file contains timing arcs that are not in the timing shell, a VHDL annotator generally halts the simulation with an error.

In general, you must regard that ensuring a match between the SDF file and the timing shell as a requirement when performing SDF annotation, although Verilog simulators are typically more flexible in these requirements. This requirement can be achieved with the use of the following:

- *Templates* on page 5-12
- *SDF remap tool* on page 5-14.

5.3.1 Templates

As described in *SDF annotation* on page 5-10, it is generally assumed that the SDF file to be annotated is a match for the timing constructs specified in the timing shell. DSMs are supplied with a template file that assists in ensuring this match. The template file, typically named *device.sdft*, is structurally an SDF file but contains no numerical information. Instead it contains the names of device timing parameters. The primary intent of the template is to serve as a definitive reference for the arcs present in the timing shell. However, this does not mean that the SDF file to be annotated has to precisely match the template file. The order of timing arcs is not usually important, and there are other subtle ways that the template file can differ from the annotated SDF:

- SDF treats setup and hold timing checks as two separate timing arcs. They are usually represented by SETUP and HOLD arcs in an SDF file, and there are two placeholders in the simulation. However, SDF also enables the use of the SETUPHOLD construct. DSMs are usually written so that SETUPHOLD can be a shorthand form of separate SETUP and HOLD arcs with no ill effects.

Note

There are some timing checks when a setup check is performed on one clock edge, a hold check on the following edge, and the data signal must be held stable in the clock-phase between them. This is sometimes referred to as a nochange timing check. Some DSMs contain these type of checks, and they can be represented in an SDF file by unpaired SETUP or HOLD arcs. You must not replace unpaired arcs with a SETUPHOLD construct because this causes the simulator to try and annotate a timing check that does not exist in the timing shell. It results with an error.

-
- SDF enables the specification of single values, or triple values that represent minimum, typical, and maximum times. Templates always use the triple form for their placeholders but this is not a requirement. In addition, IOPATH entries in the template can contain up to 12 sets of placeholders but there is no requirement to provide 12-value SDF for annotation. The placeholders in an SDF template are of secondary importance to the structure of the file, and the actual SDF to be annotated can contain up to six values, representing the six possible transitions a signal can make between the three logic values, **1**, **0** and **Z**. ARM tools that generate SDF from template files, *sdffemap* and *sdffgen*, write no more than six values for an IOPATH, regardless of the contents of the template. It is also permissible for the annotated SDF to contain fewer than six values in IOPATH arcs.
 - For timing arcs that apply to vector signals, the template includes a bit-range for the arc, for example:
(IOPATH (NEGEDGE CLK) A[31:0] ...

Note

. . . indicates more line data but not relevant to the subject.

In effect, this is a shorthand way of representing 32 different timing arcs:

```
(IOPATH (NEGEDGE CLK) A[31] ...
(IOPATH (NEGEDGE CLK) A[30] ...
(IOPATH (NEGEDGE CLK) A[29] ...
(IOPATH (NEGEDGE CLK) A[28] ...
- - -
```

Note

- - - indicates more lines of SDF.

When generating an SDF file for annotation, it is acceptable to use separate arcs for individual bits of a vector signal, in addition to bundling-up ranges, or using a mixture:

```
(IOPATH (NEGEDGE CLK) A[31:24] ...
(IOPATH (NEGEDGE CLK) A[23] ...
(IOPATH (NEGEDGE CLK) A[22] ...
- - -
```

Even though you can omit certain arcs from an SDF file if you do not want them to be annotated, it is a requirement for VHDL that if a timing arc involving a vector is being annotated, then both ends of the vector must be mentioned in the SDF file or it fails to annotate. Verilog simulators usually have no such requirement.

To summarize, you must ensure that the SDF annotated on to the model is a good match for the timing shell. The supplied SDF template file provides the definitive reference for the complete set of timing arcs and the form they must take. The following set of guidelines assists you in the annotation process:

- Timing arcs that are more general versions of the same arcs in the template usually annotate anyway when using Verilog, but not VHDL. This means that you can omit edge specifiers and COND qualifiers where present in the template, and the SDF still annotates correctly in Verilog.
- Not all arcs present in the template have to be present in the SDF file to be annotated. Arcs that are not present in the SDF, but are present in the template, default to the zero-delay behavior of the model. Parameters for timing checks that have no value annotated onto them also default to zero.
- There must be no arcs present in the section of the SDF file belonging to the model that are not present in the template. This does not include INTERCONNECT or PORT delays. See *SDF remap tool* on page 5-14 for more details.

5.3.2 SDF remap tool

If you have problems with a generated SDF file not annotating because it does not provide a good match for the template then you can use the `sdfremap` tool, provided with the model. `sdfremap` reads an SDF file containing data for a complete system and a template file and rewrites the cell pertaining to the model so that it matches the template in structure.

The tool works through the template, line by line. For each arc in the template, it searches for an arc in the SDF file that matches the type of arc under consideration and uses the same signal or signals. Because `sdfremap` is more flexible than an SDF annotator, it considers arcs that would otherwise be rejected.

If multiple possible matches for an arc are found then `sdfremap` selects between them by comparing them to see which is the closest match. However, this might not always resolve the situation. Consider the situation where the template contains the following:

```
(SETUPHOLD (POSEDGE CLK) IN (Ts:Ts:Ts) (Th:Th:Th))
```

and the SDF file has the following two entries:

```
(SETUPHOLD (POSEDGE CLK) (POSEDGE IN) (5) (2))
(SETUPHOLD (POSEDGE CLK) (NEGEDGE IN) (4) (2))
```

In this case, the timing software that produced the SDF splits one setup-hold check into two arcs by considering signal **IN** rising and **IN** falling separately. This does not annotate because the template, and therefore the timing shell is less specific than the provided SDF. If you reverse the situation, annotation works on a Verilog simulator but not on a VHDL simulator, because the timing shell is designed for a single setup-hold check that does not take the transition type of **IN** into account.

In this case, `sdfremap` considers both arcs as being an equally good match for the arc in the template, based on the arc type, signals, and edge information. In this situation, it selects the most pessimistic arc from the SDF file. The selected output for this arc is:

```
(SETUPHOLD (POSEDGE CLK) IN (5) (2))
```

To provide a proper audit trail, `sdfremap` writes a log file that explains for each arc in the template:

- which arcs, if any, it considered
- what decisions it made.

In addition to correcting edge specifications, `sdfremap` also adds any `COND` entries that are present in the template and removes extra constructs that are not necessary for annotation. This might not be necessary for Verilog, but has no harmful effect.

The SDF file can specify a timing arc between two vectors, for example:

```
(IOPATH IN[31:0] OUT[31:0] ...
```

Note

The value of 31 for IOPATH is an example.

Typically this refers to a combinatorial path where **IN[0]** drives **OUT[0]**, **IN[1]** drives **OUT[1]**, and continues in this manner. As described in *Templates* on page 5-12, vector ranges in SDF are shorthand for a set of timing arcs for each bit in the range. Expanding the IOPATH, initially produces the following:

```
(IOPATH IN[31:0] OUT[0] ...
(IOPATH IN[31:0] OUT[1] ...
.
.
.
(IOPATH IN[31:0] OUT[30] ...
(IOPATH IN[31:0] OUT[31] ...
```

However, because the input half of the IOPATH is also a vector, each one of the arcs is also a shorthand for 32 more arcs:

```
(IOPATH IN[0] OUT[0] ...
(IOPATH IN[1] OUT[0] ...
.
.
.
(IOPATH IN[31] OUT[0] ...

(IOPATH IN[0] OUT[1] ...
(IOPATH IN[1] OUT[1] ...
.
.
.
(IOPATH IN[31] OUT[1] ...
```

This procedure continues, leading up to 1024 (32 squared) arcs in total. The assumption is that each bit on the output vector can be driven by any of 32 bits on the input vector. This is generally not the case, however, and the DSM model implements only 32 arcs, not 1024. In this case the template reads:

```
(IOPATH IN OUT[31:0] ...
```

Here, **IN** is now represented as a single signal so that only 32 timing arcs are inferred by the simulator, giving the required behavior. One common misunderstanding is that this representation somehow results in the wrong behavior because it is specifying the entire vector, **IN**, as the causal signal. However, the timing shell does not specify behavior, only timing is specified. See *Addition of timing shell* on page 5-4 and *Multiple*

timing paths on page 5-8 for more information. If, for example, the model drove OUT[5] because IN[5] has changed, then the timing shell recognizes this as the path, IN -> OUT[5], because if one bit of **IN** has changed, then the vector signal itself has also changed and the timing shell functions as required.

Note

See *Vector to vector IOPATH arcs* on page 5-27 for details of difficulties that can arise from vector->vector IOPATHs during annotation in VHDL simulations.

The precise behavior of sdfremap is configurable through its preferences files. You must be aware that the default behavior of the tool might not produce useful results, depending on the input, you must read through the supplied UNIX man page for sdfremap before using the tool. Common reasons for problems include:

- The tool does not find any cells to remap. sdfremap matches on the cell type entry in the SDF file. If the cell type in the input SDF does not match the cell type used in the template file, the tool is unable to recognize the cell as the one it is interested in. In this case, you can use the `celleq` facility in the preferences file to make sdfremap aware of the correct cell type. See the UNIX sdfremap man page for details of how to use the preferences file.
- The remapped SDF refers to the wrong hierarchy level. At the time of writing, DSMs place the timing shell one level below the top-level of their wrapper. Most SDF files are generated with the expectation that the timing shell is at the level at which the DSM is instantiated. In the future, DSMs might be generated with the timing shell at this level, resolving this problem. Presently, you can resolve this issue by using the `pathtrails` facility in the sdfremap preferences file.

Note

The use of the `pathtrails` facility does not fix the hierarchy in any SDF INTERCONNECT expressions, because these are specified outside the cell that they reference and are therefore untouched by sdfremap. Currently, hierarchy problems with interconnect delays either require resolving manually or by using a text processing tool such as the UNIX `sed` utility.

- sdfremap does not correctly choose the most pessimistic timing arc. There are a number of reasons why this happen. For input checks, matching on the type of check takes higher priority than pessimistic time matching, so the tool might select a more optimistic timing check that is a closer match to the format of the template over a more pessimistic check that is a poor match.

For IOPATHs that can contain up to six sets of values, sdfremap regards the most pessimistic delay as being the one with the highest average (mean) of the values given.

For three-value SDF, sdfremap defaults to calculating what arc is the most pessimistic by considering the typical timing values, that is, those in the middle of the value-triples. An unhelpful case is where SDF files contain empty typical values for all triples, for example:

```
(SETUPHOLD (POSEDGE CLK) IN (1::2) (2::2))
```

In this case, the default behavior of sdfremap effectively turns pessimistic value matching off, because it regards all timing arcs as specifying zero as their value. You can configure sdfremap to use any of the three values for the purpose of calculating pessimistic delays through the use of the preferences file. See the UNIX sdfremap man page for more information.

5.4 Interconnect delays

The timing checks and delays explicitly built in to a timing shell and referenced in an SDF template file are applied at the boundary of the model and do not interact directly with other cells in the design.

The timing shells also have provision for interconnect delays from neighboring cells to be annotated. In the simple case then interconnect delays are modeled in HDL simulators by placing a delay buffer on the input side of a wire. Timing shells place a buffer on every input to the model. As with IOPATH delays, the buffer has a placeholder-delay associated with it that is initially set to zero.

Note

The simple case is a single-source interconnect delay. This is if an interconnect wire between cells has a single driver and a single reader.

During SDF annotation, the placeholder is overridden by SDF INTERCONNECT or PORT timing arcs, in the simple case, PORT and INTERCONNECT are treated identically by SDF annotators. Any input signals that are subject to interconnect delays are then held by the buffer for an appropriate amount of time. After this delay, the event is registered by any timing checks and the model itself. Interconnect delay buffers can therefore be regarded as an extra shell around the timing shell that delays input events before passing them through to the normal timing shell where they are processed as usual. Setup and hold, in addition to any other timing checks, occur on the delayed value of the input signal, not the non-delayed version. If a signal used as the reference signal in an IOPATH delay is subject to an interconnect delay, it is the delayed version of the event that is regarded as causal.

Output events driven by the model propagate into other components within the wider design which can also choose to add interconnect delays in addition to any IOPATH delay added by the timing shell of the model. These delays are applied downstream by whatever is consuming the events, and do not affect the timing shell.

5.4.1 Multi-source interconnect delays

In some cases, an input to the model can have more than one possible driver, such as a databus that can be driven by memory or a DMA peripheral. In such cases, a single value for the interconnect delay, annotated at the site where the signal is read, is insufficient because the interconnect delay can differ according to the driver. See the section on interconnect delays in OVI SDF2.1, for an example. See *Other publications* on page xii.

In the case of these multi-source interconnect delays, Verilog SDF annotators, when invoked with the correct options attempt to spread the delays around, see your simulator vendor manual for more information. They annotate part of the delay on the input part of the signal, and part of the delay on the output buffers where the signal is driven. ARM Verilog timing shells function with multi-source interconnect delays if the models driving them have the appropriate output buffers. ARM Verilog timing shells also provide the appropriate buffer constructs on their own outputs to enable you to use them as annotation sites for multi-source interconnect delays.

ARM VHDL timing shells only support single-source interconnect delays.

5.5 Use of negative timing checks

The simple zero delay behavior outlined in *Zero delay model core* on page 5-4 is generally suitable for simulating the typical case. Synchronous inputs are required to be stable at some point before a clock edge, the setup time, and held stable for some time afterwards, the hold time. Figure 5-6 shows that sometimes a setup time can be negative.

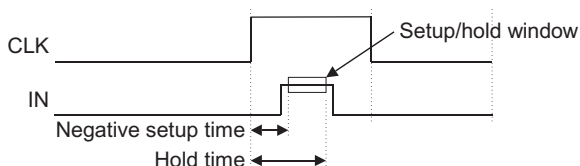


Figure 5-6 Negative setup time

In this case, the input **IN** signal is permitted to change for some time after the clock edge when it is nominally sampled. This can occur because of clock-tree distribution delays inside the device. The clock reaches some parts of the hardware before it reaches others. In these cases, the logic inside the device that is working with these inputs is effectively using a delayed version of the clock. The timing behavior of the device, as specified in an SDF file, is still relative to the boundary of the device. Relative to the clock as seen at the device boundary, the setup time of the input, **IN**, is negative.

The zero-delay model with a boundary timing shell cannot directly handle this situation because it does not use distributed delays inside the device directly, pin-to-pin timing annotation is not possible if it does, but models them by delaying a driven output by the total of all the delays involved in producing it. This cumulative delay is the value used in **IOPATH** directives in the SDF file.

Any behavior in the model that depends on the sampled value of **IN**, but is determined as a response to the clock edge therefore runs the risk of sampling the wrong value of **IN** and diverging from the behavior of the real device.

A solution to this problem is for certain events to be delayed inside the timing shell, before they are presented to the model. In this case, the clock signal is delayed to produce a new signal named **CLK'**. Figure 5-7 on page 5-21 shows this.

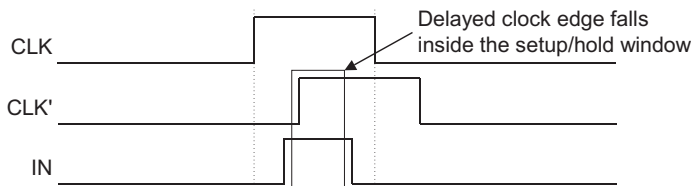


Figure 5-7 Delaying a clock signal to produce a new signal

Timing checks are still performed relative to the original clock, but the model sees the delayed version, and therefore samples **IN** at a time when it is known to be stable inside its setup and hold window.

In isolation, this is all that is required. However, the clock is delayed from the perspective of the model and therefore any outputs driven from that clock are also delayed. If the time that the clock is delayed is subtracted from all IOPATH times from the clock to any outputs, the behavior outside the timing shell is still correct. In addition, delaying the clock can potentially move the point when other synchronous inputs are sampled to beyond the end of their hold time. Figure 5-8 shows how this occurs.

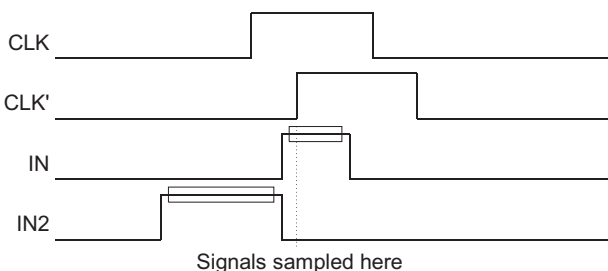


Figure 5-8 Inputs sampled beyond hold time

To accommodate this, adding a delay to the clock means that IOPATH delays must be reduced by a corresponding amount. Also, other input signals might require a delay applying to them, similar to the clock delay, so that they are sampled within their setup and hold windows.

The Verilog SDF annotator provides a facility where it automatically delays input signals by an appropriate amount and adjust the output delays to compensate. This is typically enabled with an appropriate switch to the Verilog simulator when it is invoked. See your simulator manual for more information. If this switch is not specified then negative timing checks are typically set to zero.

ARM timing shells for Verilog simulators are constructed in such a way that the simulator can perform this adjustment for negative timing constraints. Negative timing checks can be used with Verilog models because of this. However, there can be cases where it is not possible to adjust the timing behavior to ensure that all IOPATHs and timing checks function as defined because there can be an IOPATH delay that is smaller than the amount that the clock requires to be delayed to accommodate a negative setup time. Some timing checks might still be overly-pessimistic because of this.

ARM VHDL models do not support negative timing checks.

5.6 STA and HDL annotated simulation differences

This section describes any potential differences between the STA results of a design that uses an ARM core and the timing behavior of its DSM during dynamic HDL simulation of the design. An STA model of a core represents the timing reference model of that core and the timing shell of the DSM must match it as faithfully as possible.

The STA timing model, the Synopsys library view, of an ARM core is a pin-to-pin black-box timing model. These models contain timing arcs and lookup tables that calculate the values used in the timing arcs, given the context of the usage of the core in a system. Consequently, the STA tool has no knowledge of the internal structure of the core. This is the same situation for the HDL timing shell provided with a DSM. Therefore it is theoretically possible for the timing shell to contain an exact equivalent set of timing arcs to the STA model, although for some models this is not the case.

There are various reasons why the timing arcs might not match between the two model types and these are described in *Missing arcs*. There are also cases where the timing results can be different because limitations of HDL simulators with respect to STA, these are described in *HDL simulation limitations* on page 5-24. Some mismatches between STA and HDL simulation can result in the simulation timing behavior being more optimistic than STA and others cause it to be more pessimistic. In the optimistic case, an HDL simulation of a design does not report timing violations found by STA assuming the failing arcs are exercised during the simulation. In the pessimistic case, an HDL simulation issues false timing errors and therefore, error free, full speed simulation is not possible.

5.6.1 Missing arcs

In this case timing arcs that are present in the STA model of a core are not in its DSM timing shell. This causes the DSM to be more optimistic than the STA model. This can be caused by:

- Version mismatches.
- Output to output delays.
- Output setup and hold checks.
- A DSM not matching the implementation of a synthesizable core from a core-licensee. These are cores with the -S suffix and ETMs.

Version mismatches

In some cases, a core has had a design change that results in timing arcs being added to the STA model and equivalent changes were not made to its DSM timing shell. This can be remedied by a minor update to the DSM when it is identified.

Output-to-output delays

In some cases implementations of synthesizable devices have output signals that are directly fed-back internally and pass through some logic that drives another output pin. Currently these arcs are not implemented in DSMs. The core-licensee can solve this problem by resynthesizing the device using the `set_fix_multiple_port_nets` command during synthesis with Synopsys tools.

Output setup and hold checks

This is similar to output to output delays and also applies to synthesizable devices. It occurs when output signals are directly fed back internally to flip-flop inputs. You can resolve this problem in the same manner that resolves the output-to-output delays.

DSM does not match the implementation of a core from a core-licensee

This is an issue for synthesizable devices only. The timing arcs in the DSMs for these cores are developed from test chip implementations of the associated cores. When synthesizable devices are implemented by core-licensees, arcs can vary slightly from implementation to implementation because of modeling differences between technology libraries, synthesis methodologies, and tool versions used. For synthesizable devices, silicon vendors generate their own STA view that matches their implementation of a core, that ARM Limited has no knowledge of. The solution to this is for a core-licensee to also generate the DSM timing shell for their implementation of a core.

5.6.2 HDL simulation limitations

These are related to how the SDF data is treated in the simulation back-annotation flow. These limitations lead to the simulation being more pessimistic than STA. These can occur in negative timing checks, rising and falling setup and hold checks, and single negative setup or hold checks.

Negative timing check issues

This is described in *Use of negative timing checks* on page 5-20 where the output delay for some ports can be smaller than the amount that the clock is required to be delayed. HDL simulators honour the output delay requirements and zeros any conflicting negative setup checks. STA does not have these constraints and can consider each arc individually, but simulation must take into account the relationships between output delays and setup and hold checks.

Rising and falling setup and hold checks

As described in *SDF remap tool* on page 5-14, the `sdfremap` tool combines edge specific pairs of setup and hold statements in SDF files into single non-edge specific statements. In doing so it chooses the most pessimistic values from the original file. Synthesis and STA uses the rising and falling values independently. The implemented design can rely on the fact that there is more setup margin for the rising edge of an input verses the falling edge and also in the reverse case. This can cause a simulation to falsely report a setup or hold violation.

Note

This only applies to edge-specifiers on the data signal. Clock-edge specific timing checks are implemented in the timing shell.

Single negative setup or hold checks

In rare cases, the STA model can contain a single setup check without a corresponding hold check or a hold check without a corresponding setup check. This is not a problem if the check value is positive but if it is negative then the check is set to zero in simulation. This occurs because only compound Verilog `$setuphold` checks can be annotated with negative values and if one of the pair of values is missing then the default value in the timing is used, which is zero. Consequently, one half of the check is negative and the other zero. This is invalid because the sum of the two values must be positive. The Verilog simulator overrides the value in the SDF file and sets it to zero. It also issues a warning message informing you that it has set the timing check value to zero.

5.7 Limitations of the timing model

The discussion of limitations of the timing model is subdivided into:

- *IOPATH retain*
- *Vector to vector IOPATH arcs* on page 5-27.

5.7.1 IOPATH retain

Previously, transitions on delayed outputs have been represented as a single transition from the old value to the new value at a point in simulation time. In reality, the latest time when the old value is guaranteed to be valid and the earliest time when the new value is guaranteed to be valid can be separated by a period of uncertainty, where the value is unknown as shown in Figure 5-9.

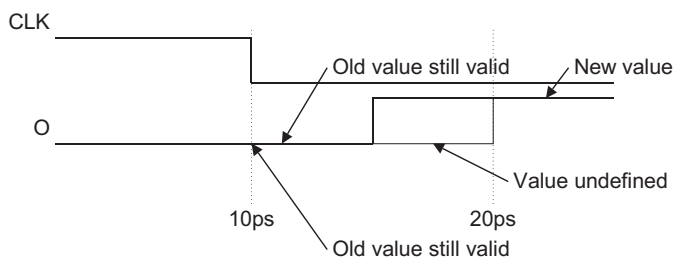


Figure 5-9 Uncertainty of value

In a simulation, this behavior is modeled by driving the output to X at 15ps, and to 1 at 20ps.

The delays in a DSM must operate within the limits of a timing shell, and so this behavior cannot be modeled directly. All delays are applied by the code in the timing shell and the model drives the output directly to the new value at the time when the model is invoked. Any modeling of the unknown-period must be performed in the timing shell.

The SDF file format contains support for specifying the time that the previous value of an output remains valid for, as distinct from the time when the new value becomes valid. This facility is called *IOPATH RETAIN* and enables up to three times to be specified for the retain time. This is the time that the previous value is known to be valid. These three times represent a transition from HIGH, LOW, and high-impedance respectively.

At the time of writing, it is understood that none of the Verilog simulators supported by ARM Limited make use of the *IOPATH RETAIN* facility during SDF annotation. It is ignored. Verilog models do not support retain times on output delays because of this.

VHDL has recently added some support for IOPATH RETAIN in its SDF annotator, but the way this is accomplished requires the timing shell to know how many delay values, up to six, that the corresponding IOPATH entry in the SDF file is going to provide at the time the timing shell is generated. Because of this limitation, ARM VHDL timing shells do not presently support IOPATH RETAIN.

Simulation inaccuracies

An important implication of the lack of support for IOPATH RETAIN is that you must choose whether to put the transition to the new value at the point when the old value ceases to be valid, or at the point when the new value is known to be valid when performing SDF back-annotation.

If a device that uses the output values from the DSM samples one during the period when the value is not defined, it either gets the old value or the new value, depending on which of the two possible times for the transition is in use. In reality, the values must not be sampled during this period, and support for IOPATH RETAIN ensures that anything sampled in that period reads an X.

In the absence of support for IOPATH RETAIN this situation can potentially lead to a divergence between the simulation and the behavior of the physical device. Furthermore, this divergence is not reported with a warning message, the simulation silently uses the wrong value. One way of detecting this situation is to perform two simulations, one using an SDF file containing minimum values and the other using an SDF file containing maximum values. The SDF files must be generated by an STA tool using the minimum and maximum timing views of a device, respectively. The results of the two simulations are expected to be identical provided outputs are not sampled in the undefined period.

5.7.2 Vector to vector IOPATH arcs

VHDL, through the 1995 VITAL standard, mandated an annotation scheme for IOPATHs where both signals are vectors, which requires annotation placeholders for the cartesian product of the two vectors. For example, two 32 bit vectors requires 32^2 (1024) annotation points. As described in see *SDF remap tool* on page 5-14, this is inefficient so ARM SDF template files represent one of these signals as a scalar.

The newer, VITAL 2000 standard, recognized that the earlier mechanism was not suitable in situations where there is a one-to-one relationship between an element on the input vector and the same element on the output vector:

- **IN[0]** drives **OUT[0]**
- **IN[1]** drives **OUT[1]**
- ...

This being the case, newer VHDL simulators permit a special case where both input vectors are the same size.

Note

Some devices have vector to vector combinatorial paths where both vectors are not the same size, and this can cause problems with some VITAL 2000 compliant simulator versions.

ARM Limited is investigating ways to work around this difficulty. If you do have a problem with annotation of vector to vector IOPATHs on a VITAL 2000 compliant simulator, the current recommended work-around is to use a previous version of the simulator which implements VITAL 95 only.

This problem does not affect Verilog simulators.

Appendix A

EIS Output Format

This appendix describes the format of the `log.eis` files written out by DSMs. It contains the following sections:

- *About the EIS file format* on page A-2
- *ARM7TDMI r4 format* on page A-3
- *ARM7TDMI r4 example* on page A-5.

Note

This appendix specifically describes the output format produced by the ARM7TDMI r4 DSM. it can be used as a guide to the similar `log.eis` format of other ARM7 and ARM9 DSMs.

This appendix does not cover models of newer ARM cores such as ARM11 DSMs. These use a different trace file format.

The typographical conventions used in this appendix are:

- angled brackets, `< >`, enclose field names
 - square brackets, `[]`, enclose optional fields.
-

A.1 About the EIS file format

DSMs produced by ARM Limited represent the functionality of ARM devices. These models incorporate functionality to represent the behavior of the device plus additional features to assist you to understand and debug simulation runs. One such feature is the production of a trace file in human-readable form that shows the sequence of instructions executed in the device with additional details of used data values. This capability is called executed instruction trace and the data file is usually referred to as the `log.eis` because of its default file name.

DSMs of ARM devices usually include this trace capability. The exact format can vary between devices or from versions. This appendix describes some specific examples of the format to provide a basis for understanding any other variations of the format.

DSMs of newer ARM devices such as ARM11 processors adopt a different format that is based on ARMulator trace format. This format is described in the *RealView ARMulator ISS User Guide*.

A.2 ARM7TDMI r4 format

The ARM7TDMI r4 executed instruction trace is a simple format that incorporates information about:

- which instruction was processed
- whether it was executed or skipped
- where the instruction was stored
- any external address and data that was referenced.

The format includes two types of line that are described in:

- *Instruction description format*
- *Memory description format* on page A-4.

A.2.1 Instruction description format

The format of the instruction description line is:

```
<cycle>    [CCFAIL]    <I-address>    <Opcode>    <Disass>    [; <comment>]
```

The fields are defined as follows:

<cycle>	This is a decimal number that represents the clock cycle count in which the instruction reached the execution stage in the device pipeline. A DSM model can provide a traceable register to give the corresponding clock count, this enables you to correlate log.eis trace against simulation waveforms.
CCFAIL	This is an optional literal text field. Most ARM instructions are conditionally executable, meaning that the instruction code contains a field specifying various conditions that control whether the instruction is executed or not. These conditions are evaluated when the instruction reaches the execution stage of the ARM pipeline: • if the conditions are met then the instruction is executed and the log.eis field is left blank. • if the conditions are not met then the instruction is not executed and the CCFAIL text is logged.
<I-address>	This is a hexadecimal number which indicates the address that this instruction was fetched from.
<Opcode>	This is a hexadecimal number that shows the instruction code that is being executed. You can investigate the meaning of individual bits by referencing the <i>ARM Architecture Reference Manual</i> .

<Disass> This is a disassembly listing of the instruction defined by <Opcode>. Assembler instruction names and formats are described in the *ARM Architecture Reference Manual*. Instructions that operate on addresses offset from the current value of the *Program Counter* (PC), known as PC-relative, are represented conveniently as {pc}+/-<hex value>.

———— **Note** —————

The PC is always two instructions ahead of the currently executing instruction.

<comment> To avoid confusion this field gives the actual absolute hexadecimal address given by a PC-relative offset or the calculated result of an immediate shifter operand.

A.2.2 Memory description format

Each load or store instruction description line in the log.eis trace is immediately followed by a memory description line that gives information about the address and data involved in that memory access. The format of the memory description line is:

<cycle> Data <direction> [<size>] <D-address> <value>

The fields are defined as follows:

<cycle> This is a decimal clock cycle count of the cycle in which the memory access took place.

Data Literal text.

<direction> Read to indicate a read from memory or Write to indicate a write to memory.

<size> This is a text description that indicates if a data operation was performed using a byte, 8 bits, or halfword, 16 bits. The default is a word, 32 bits, and is represented by a blank.

<D-address> This is a hexadecimal value that indicates the memory address accessed in this instruction.

<value> This is a hexadecimal value that indicates the value read from or written to memory.

A.3 ARM7TDMI r4 example

Example A-1 shows a portion from an ARM7TDMI DSM log.eis file:

Example A-1

24		00000050	e3c12003	BIC	r2,r1,#3		
25		00000054	e2110003	ANDS	r0,r1,#3		
26		00000058	e3a017c0	MOV	r1,#0xc0, 14 ; #0x30000000		
27		0000005c	e492c004	LDR	r12,[r2],#4		
28				Data Read		00000094	202a2a0a
30	CCFAIL	00000060	108ff180	ADDNE	pc,pc,r0,LSL #3		
31		00000064	e1a00000	NOP			
32		00000068	e013000c	ANDS	r0,r3,r12		
33		0000006c	15c10000	STRNEB	r0,[r1,#0]		
34				Data Write Byte		03000000	0a
35		00000070	1013042c	ANDNES	r0,r3,r12,LSR #8		
36		00000074	15c10000	STRNEB	r0,[r1,#0]		
37				Data Write Byte		03000000	2a
38		00000078	1013082c	ANDNES	r0,r3,r12,LSR #16		
39		0000007c	15c10000	STRNEB	r0,[r1,#0]		
40				Data Write Byte		03000000	2a
41		00000080	10130c2c	ANDNES	r0,r3,r12,LSR #24		
42		00000084	1492c004	LDRNE	r12,[r2],#4		
43				Data Read		00000098	54534554
45		00000088	15c10000	STRNEB	r0,[r1,#0]		
46				Data Write Byte		03000000	20
47		0000008c	1affffff5	BNE	{pc}-0x24 ; #0x68		
50		00000068	e013000c	ANDS	r0,r3,r12		

Cycle 26 shows a register being loaded through a constant, 0xC0, with an immediate, implied, rotate right of 14 places, and the comment field indicates that C0 rotated right by 14, that is left by 18, yields 0x03000000.

Cycle 30 shows a conditional ADD being skipped because the condition is not met. The conditional store at cycle 33 has met its condition and is therefore executed with a memory description line showing the byte value 0x0A written to address 0x03000000.

Cycle 47 shows a conditional branch at address 0x008C being taken and consequently a 3-cycle penalty occurs because the pipeline is flushed and refilled to execute the branch target instruction at address 0x0068 in cycle 50.

Glossary

This glossary describes some of the terms used in this document. Where terms can have several meanings then the meaning provided in this glossary is intended.

Back-annotation	The process of applying timing characteristics from the implementation process onto a model.
Boundary scan chain	A boundary scan chain is made up of serially-connected devices that implements boundary scan technology using a standard JTAG TAP interface. Each device contains at least one TAP controller containing shift registers that form the chain connected between TDI and TDO , through which test data is shifted. Processors can contain several shift registers to enable you to access selected parts of the device.
Clock gating	Gating a clock signal for a macrocell with a control signal, and using the modified clock that results to control the operating state of the macrocell.
CRF	<i>See</i> Condensed Reference Format.
Condensed Reference Format (CRF)	An ARM proprietary file format for specifying test vectors.
Delta cycle	A simulation cycle in which the simulation time at the beginning of the cycle is the same as at the end of the cycle. That is, simulation time is not advanced in a delta cycle.

Design Simulation Model (DSM)

A back-annotation-capable simulation model that can be included within a range of target HDL simulators. It consists of a functional core block and a Verilog or VHDL wrapper.

Delta-sweeping

The process by which the VHDL simulator advances through delta cycles. A sweep covers many delta cycles.

Direct Memory Access

An operation that accesses main memory directly, without the processor performing any accesses to the data concerned.

DVS

See Device Validation Suite.

Device Validation Suite

A set of tests to check the functionality of a device against the functionality defined in the Technical Reference Manual. Also stresses Bus Interface Unit (BIU), and low-level memory sub-system, pipeline, cache and Tightly Coupled Memory (TCM) behavior.

Internal scan chain

A series of registers connected together to form a path through a device, used during production testing to import test patterns into internal nodes of the device and export the resulting values.

Model Manager

A software control manager that handles the event transactions between the model and simulator.

Programming Language Interface (PLI)

For Verilog simulators, an interface by which code written in a different language can be included in a simulation.

SDF

See Standard Delay Format.

Sign-Off Model (SOM)

An opaque, compiled simulation model generated from a technology specific netlist of a core, derived after gate level synthesis and timing annotation, that you can use in back-annotated gate-level simulations to prove the function and timing behavior of the device. A SOM is supplied by the creator of the implementation of the core. This might or might not be ARM Limited. A SOM enables accurate timing simulation of SoCs and simulation using production test vectors from *Automatic Test Pattern Generation* (ATPG) tool such as Synopsys TetraMAX. It only supports back-annotation using SDF files. The SOM includes timing information but provides slower simulation than a DSM.

SOM

See Sign-Off Model.

Standard Delay Format (SDF)

The format of a file that contains timing information to the level of individual bits of buses and is used in SDF back-annotation. An SDF file can be generated in a number of ways, but most commonly from a delay calculator.

TAP

See Test Access Port.

Test Access Port (TAP)

The collection of four mandatory terminals and one optional terminal that form the input/output and control interface to a JTAG boundary-scan architecture. The mandatory terminals are **TDI**, **TDO**, **TMS**, and **TCK**. The optional terminal is **TRST**.

Index

A

- About DSMs 1-2
- Addition of timing shell 5-4
- Annotation, SDF 5-10

C

- Cache limitations 4-6
- Causal inputs, multiple 5-8
- CLK** signal 5-4
- Configuring
 - DSM 3-10
 - HDL wrapper 3-3
 - model manager 3-4
- Conventions
 - numerical xii
 - typographical xi

D

- Debugging 4-2
 - programmer's model 4-2
- Delaying a clock signal to produce a new signal 5-21
- Delays
 - interconnect 5-18
 - multi-source 5-18
- Directory structure and files 2-3
- DSMs
 - about 1-2
 - configuring 3-10
 - features 1-2
 - simulation 1-3

E

- EIS 4-3
 - about A-2
 - example A-5
 - format A-3
 - instruction description format A-3
 - memory description format A-4
- Environment variables 3-2
- Executed Instruction Stream
 - see* EIS

F

- Features of DSMs 1-2

H

HDL SDF wrapper 5-2
 configuring 3-3
 HDL simulation limitations 5-24
 High performance and negative setup
 time 5-7

I

IN signal 5-4
 Input signals sampled beyond hold time
 5-21
 Installation 2-2
 Integration 1-3
 Interconnect delays 5-18
 multi-source 5-18
 Internal scan chain modeling limitation
 4-6
 IOPATH retain 5-26

L

Licensing 2-3, 2-6
 obtaining a key 2-6
 server program 2-6
 setting up your environment 2-8
 Limitations
 caches 4-6
 internal scan chain modeling 4-6
 registers 4-6
 timing model 5-26
 unsupported simulator functions
 4-6

M

Missing arcs 5-23
 Model environment 3-2
 Model manager 1-3, 2-5, 3-2, 3-4
 about 3-4
 configuring 3-4
 configuring your simulator 3-5
 directory 2-4
 how it loads a model 3-9

ModelSim Verilog configuration
 3-7

ModelSim VHDL configuration 3-7
 NC-Verilog configuration 3-5
 NC-VHDL configuration 3-6
 VCS configuration 3-7
 Verilog-XL configuration 3-6
 ModelSim Verilog 3-7
 ModelSim VHDL 3-7
 Modified sequence of events 5-6
 Multi-source interconnect delays 5-18

N

NC-Verilog 3-5
 NC-VHDL 3-6
 Negative setup time 5-20
 high performance device 5-7
 Negative timing
 checks 5-20
 constraints 5-7
 Numerical conventions xii

O

OOB
 directory structure and files 2-3
 installation 2-2
 licensing 2-6
 running the test script 2-10
 set up your simulator 2-10
 setting up your model 3-2
 testing your model installation 2-10
Out Of Box
 see **OOB**
OUT signal 5-4
 OVI SDF2.1 5-11, 5-18

P

Pin-to-pin timing model
 high performance and negative setup
 time 5-7
 synchronous block 5-4
 Programmer's model 4-2

R

Register limitations 4-6
 Running the test script 2-10

S

SDF 1-2
 annotatable timing shell 3-3
 annotation 5-10
 COND facility 5-8, 5-9
 example file 2-4
 IOPATH construct 5-8
 SETUPHOLD construct 5-12
 template 2-4, 5-12
 templates 5-12
 timing wrapper 2-4
 tools directory 2-5
 SDF remap 5-12
 tool 5-14
 Set up your simulator 2-10
 Setting up your model 3-2
 Setup time, negative 5-20
 SETUPHOLD 5-12
 Signals
 CLK 5-4
 IN 5-4
 OUT 5-4
 Signals sampled beyond hold time 5-21
 Sign-Off Model
 see **SOM**
 Simulation
 inaccuracies 5-27
 Simulation with DSMs 1-3
 Simulators, unsupported functions 4-6
SOM xi, 1-3, 3-3, 4-6, 5-1
 STA 5-8
 STA and HDL annotated simulation
 differences 5-23
 Standard Delay Format
 see **SDF**
 State-dependent timing 5-8
 Static timing analysis
 see **STA**
 SWIFT 2-9
 Synchronous block 5-4

T

- Templates, SDF 5-12
- Test script, running 2-10
- Testing your model installation 2-10
- Timing model, limitations 5-26
- Timing paths, multiple 5-8
- Timing shell
 - addition 5-4
 - behavior 5-3
 - HDL SDF wrapper 5-2
 - structure 5-10
- Tool, SDFremap 5-14
- Typographical conventions xi

U

- Uncertainty of value 5-26
- Use of negative timing checks 5-20

V

- VCS 3-7
- Vector to vector IOPATH arcs 5-27
- Verilog-XL 3-6
- VITAL 2000 5-28
- VITAL 95 5-28

